

Experiment No. 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory

Relevance of require Statements in Solidity Programs In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data. For example, in a Decentralized Library Smart Contract, require is used to enforce critical business logic:

- **Access Control:** Checking if the person calling the addBook function is the authorized librarian (require(msg.sender == librarian, "Only the librarian can add books.")).
- **Availability:** Ensuring a book is actually available before allowing a user to borrow it.
- **Verification:** Ensuring that only the specific user who borrowed a book is permitted to return it. Thus, require statements enforce security, correctness, and reliability in smart contracts, while the attached error messages make debugging much easier.

Keywords: mapping, storage, and memory

- **mapping:** A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). In the Library system, we use mapping(uint => Book) public books;. Here, each unique Book ID (an unsigned integer) is mapped to a Book structure containing its title, author, and availability status. Mappings do not have a length property and cannot be iterated over directly, making them highly gas-efficient for direct lookups.
- **storage:** In Solidity, storage refers to the permanent memory of the contract, stored directly on the Ethereum blockchain. Variables declared at the contract level (like librarian, bookCount, and the books mapping) are stored in storage by default. Because writing to blockchain storage consumes gas, it is expensive. When a user borrows a book, we modify a storage reference so that the book's isAvailable status permanently updates on the blockchain for all users to see.
- **memory:** In contrast, memory is temporary storage used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. It is significantly cheaper than storage in terms of gas cost. In the Library contract, when passing book titles and author names into the addBook function, we use string memory because those text values only need to be held temporarily before being saved into the permanent storage mapping.

Using string for Text Flexibility While fixed-size data types like bytes32 are highly gas-efficient and optimize comparison operations, they are limited to exactly 32 characters. In an application like a Library System, book titles and author names vary significantly in length and often exceed 32 characters. Therefore, string is used. string is a dynamically sized type, meaning it can store text of variable length. While it requires more complex handling inside the Ethereum Virtual Machine (EVM) and consumes slightly more gas than bytes32, it is essential for readability, flexibility, and user-friendliness when dealing with real-world text data like book details.

Outputs

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

99.999 ETH

Deploy

Deployed Contracts 1

Transaction History

Deployed Contracts 1

+ Add Contract

Interact with a deployed contract

Library

0xd91...39138

Remix VM

2s ago

Functions

Search functions...

addBook (string, string)

_title (string)

_author (string)

Calldata

Params

Auto-Fill with AI Samples

Transact

Blockchain_Fundamentals

Vitalik

Calldata

Params

Auto-Fill with AI Samples

Transact

borrowBook (uint256)

_bookId (uint256)

Calldata

Params

Auto-Fill with AI Samples

Transact

returnBook (uint256)

_bookId (uint256)

Calldata

Params

Auto-Fill with AI Samples

Transact

bookCount

0:uint256: 1

Call

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 2

0xAb8...35cb2

99.999 ETH

Deploy

Deployed Contracts 1

Transaction History 2

Deployed Contracts 1

+ Add Contract

Interact with a deployed contract

Library

0xd91...39138

Remix VM

4min ago

Functions

Search functions...

addBook (string, string)

Hyperledger Fabric

Ved

Calldata

Params

Auto-Fill with AI Samples

Transact

library.sol

3

4

5

6

7

**

* @title Library

* @dev Implements a simple book borrowing system demonstrating structs, maps, and events.

*/

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

CAL

[call] from: 0x50380a6a701c568545dcfc803fc875f56beddC4 to: Library.bookCount()

L

data: 0xb90...5ad78

Debug

[vm]

from: 0xAb8...35cb2 to: Library.addBook(string,string) 0xd91...39138 value: 0 wei

data: 0xd8a...00000 logs: 0 hash: 0xa8a...353ea

Debug

transact to Library.addBook errored: Error occurred: Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Only the librarian can perform this action".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Only the librarian can perform this action".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Alert

Initialize as git repo

RemixAI Copilot (disabled)

Account 2

0xAb8...35cb2

99.999 ETH

Deploy

Deployed Contracts 1

Transaction History 2

Interact with a deployed contract

_author (string)

Calldata

Params

Auto-Fill with AI Samples

Transact

borrowBook (uint256)

1

Calldata

Params

Auto-Fill with AI Samples

Transact

returnBook (uint256)

_bookId (uint256)

Calldata

Params

Auto-Fill with AI Samples

Transact

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

transact to Library.addBook errored: Error occurred: Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Only the librarian can perform this action".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert

The transaction has been reverted to the initial state.

Reason provided by the contract: "Only the librarian can perform this action".

If the transaction failed for not having enough gas, try increasing the gas limit gently..

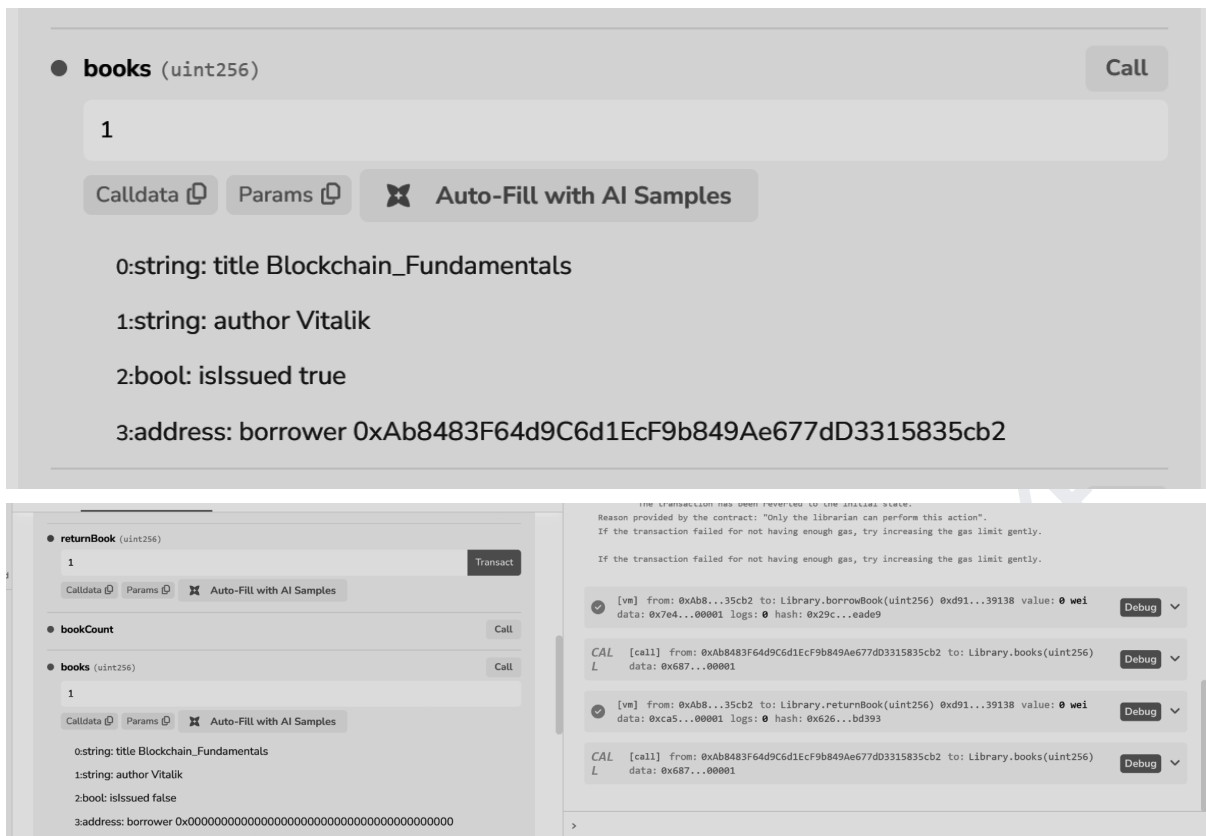
If the transaction failed for not having enough gas, try increasing the gas limit gently..

[vm]

from: 0xAb8...35cb2 to: Library.borrowBook(uint256) 0xd91...39138 value: 0 wei

data: 0x7e4...00001 logs: 0 hash: 0x29c...eade9

Debug



Conclusion

The Decentralized Library smart contract was successfully compiled, deployed, and tested using Solidity and the Remix IDE. This experiment demonstrated the practical implementation of a blockchain-based tracking system, including role-based access control (allowing only a librarian to add books) and user-driven state modifications (borrowing and returning books). Furthermore, it provided a hands-on understanding of essential Solidity concepts: utilizing require statements for robust transaction security, implementing mapping to efficiently associate IDs with complex structures, balancing gas costs using permanent storage versus temporary memory, and leveraging the dynamically sized string type to accommodate flexible text inputs.